BBK2BAB2 – ETIKA PROFESI, REGULASI TI DAN PROPERTI INTELEKTUAL

Chapter 3
Ethical Decisions in Software Development



Faqih Hamami, S.T., M.T.
Deden Witarsjah, Ph.D
Team Teaching

Sistem Informasi – Fakultas Rekayasa Industri

Pokok Bahasan

01
Strategies for Engineering Quality Software

02
The Importance of Software Quality

03
Software Development Process

04
Software Product Liability

05
Capability Maturity Model Integration

06
Development of Safety-Critical Systems

07
Quality Management Standards

Strategies for Engineering Software Quality

1

Tujuan

- ✓ Mengapa perusahaan memerlukan perangkat lunak berkualitas tinggi dalam sistem bisnis, sistem kontrol proses industri, dan produk konsumen?
- ✓ Apa masalah etika potensial yang dihadapi oleh produsen perangkat lunak dalam membuat pertukaran antara jadwal proyek, biaya proyek, dan kualitas perangkat lunak?
- ✓ Apa empat jenis klaim kewajiban produk perangkat lunak yang paling umum?

Tujuan

- ✓ Apa saja komponen penting dari metodologi pengembangan perangkat lunak, dan apa manfaat menggunakan metodologi seperti itu?
- ✓ Bagaimana Integrasi Model Kematangan Kemampuan meningkatkan proses pengembangan perangkat lunak organisasi?
- ✓ Apa itu sistem yang kritis terhadap keselamatan, dan tindakan khusus apa yang diperlukan selama pengembangannya?

Strategi untuk Rekayasa Kualitas Perangkat Lunak

- ✓ Sistem perangkat lunak berkualitas tinggi:
  - Bekerja dengan cepat dan efisien
  - Beroperasi dengan aman dan andal
  - Memenuhi kebutuhan pengguna mereka
  - Diperlukan untuk mendukung bidang:
    - Kontrol lalu lintas udara
    - Keamanan mobil
    - Kesehatan
    - Militer dan pertahanan

- ✓ Kerusakan perangkat lunak
  - Dapat menyebabkan sistem gagal
  - Dampak mungkin sepele atau sangat serius
  - Halus dan tidak terdeteksi atau sangat mencolok
- ✓ Kualitas perangkat lunak
  - Sejauh mana perangkat lunak memenuhi kebutuhan pengguna
- ✓ Pentingnya manajemen mutu

- ✓ Penyebab utama untuk kualitas perangkat lunak yang buruk:
  - Banyak pengembang tidak tahu bagaimana mendesain kualitas dengan baik
- ✓ Pengembang harus:
  - Menentukan dan mengikuti prinsip-prinsip teknik yang ketat
  - Belajar dari kesalahan masa lalu
  - Memahami lingkungan operasi sistem

The Importance of Software Quality

2

Pentingnya Kualitas Software

Software yang cacat atau memiliki bugs dapat berdampak negative bahkan sangat berbahaya. Berikut contoh nyata:

Nest Thermostat

- Nest thermostat adalah perangkat pintar yang memungkinkan pengguna mengontrol pemanas rumah dari jarak jauh melalui aplikasi di ponsel.
- **Cacat/Bug:** Cacat perangkat lunak saat cuaca dingin menyebabkan perangkat ini mati atau *offline* bagi banyak pelanggan.
- **Risiko/Kerugian:** Rumah-rumah menjadi tidak memiliki pemanas, berisiko menyebabkan pipa membeku dan membahayakan kesehatan lansia, orang sakit, dan bayi.

Pentingnya Kualitas Software

Software yang cacat atau memiliki bugs dapat berdampak negative bahkan sangat berbahaya. Berikut contoh nyata:

U.S. Customs and Border Protection

- Sistem yang digunakan oleh lembaga untuk memproses para pengunjung yang memasuki Amerika Serikat di bandara.
- **Cacat/Bug:** Pembaruan perangkat lunak yang salah menyebabkan sistem lembaga ini mati total.
- **Risiko/Kerugian:** Sistem yang mati menyebabkan antrean dan penundaan besar-besaran di bandara, merusak akhir liburan banyak pelancong dan menimbulkan kekacauan luas.

Pentingnya Kualitas Software

Software yang cacat atau memiliki bugs dapat berdampak negative bahkan sangat berbahaya. Berikut contoh nyata:

Blue Cross and Blue Shield of North Carolina

- Sistem perangkat lunak yang digunakan untuk pendaftaran pelanggan, penagihan, dan pemrosesan klaim asuransi kesehatan.
- **Cacat/Bug:** Masalah perangkat lunak menyebabkan ribuan pelanggan ditagih berlebihan, kehilangan jaminan, atau terdaftar dalam paket yang salah.
- **Risiko/Kerugian:** Pelanggan ditagih lebih mahal atau tidak memiliki asuransi, dan hampir 100 penyedia layanan kesehatan tidak dapat menagih layanan mereka selama berbulan-bulan.

Pentingnya Kualitas Software

Software yang cacat atau memiliki bugs dapat berdampak negative bahkan sangat berbahaya. Berikut contoh nyata:

Analisis Pemindaian Otak fMRI

- Perangkat lunak yang digunakan untuk menganalisis data dari *fMRI* untuk membuat gambar yang menunjukkan aktivitas otak.
- **Cacat/Bug:** Cacat pada perangkat lunak analisis sering kali menyebabkan hasil *false positive*, yang menunjukkan adanya aktivitas otak padahal sebenarnya tidak ada.
- **Risiko/Kerugian:** Cacat perangkat lunak ini berpotensi membatalkan hasil dari puluhan ribu makalah dan buku ilmiah, menimbulkan kontroversi dan keraguan pada banyak hasil penelitian.

- ✓ Pertanyaan etis
  - Berapa banyak usaha dan uang untuk diinvestasikan untuk memastikan perangkat lunak berkualitas tinggi?
  - Apakah produk dapat menyebabkan kerusakan dan seperti apa dampak hukum jika mereka lakukan?

Software Development Process

3

Proses Pengembangan Perangkat Lunak

- ✓ Adalah proses membagi pekerjaan pengembangan perangkat lunak menjadi beberapa fase yang berbeda.
- ✓ Disebut juga **software development life cycle** (**SDLC**)
- ✓ Beberapa metode antara lain: *waterfall*, *prototyping*, *iterative and incremental development*, *spiral development*, *rapid application development*, *agile*, dan *extreme programming*.

Peran di Pengembangan Perangkat Lunak

- ✓ System analysts
- ✓ Programmers
- ✓ Database specialists
- ✓ Project managers
- ✓ Documentation specialists
- ✓ Trainers
- ✓ Testers

Quality assurance (QA)

- ✓ QA merupakan serangkaian kegiatan yang terencana dan sistematis yang memastikan bahwa proses dan produk software sesuai dengan persyaratan, standar, dan prosedur
- ✓ Tujuannya untuk menjamin operasi produk yang andal
- ✓ beberapa organisasi menggunakan QA sebelum produk dipasarkan dengan hanya mengandalkan pengujian saja

Software Testing

- ✓ Dynamic Testing
  - Pemeriksaan kode dengan mengeksekusi kode
- ✓ Beberapa tipe testing lainnya antara lain:
  - Static testing
  - Integration testing
  - System testing
  - User acceptance testing

Software Product Liability

4

Tanggung Jawab Software

- ✓ Product liability
  - - Tanggung jawab produsen, penjual, dan lainnya untuk cedera yang disebabkan oleh produk yang cacat
- ✓ Strict liability
  - - Terdakwa bertanggung jawab atas cedera tersebut
  - - Terlepas dari kelalaian atau niat

Strict Liability

- ✓ Penggugat harus membuktikan hanya bahwa produk perangkat lunak itu cacat atau berbahaya dan menyebabkan cedera
- ✓ Semua pihak dalam rantai distribusi bertanggung jawab
    - Pabrikan
    - Subkontraktor
    - Distributor

Negligence

- ✓ Kelalaian adalah kegagalan untuk melakukan apa yang akan dilakukan oleh orang yang berakal, atau melakukan sesuatu yang tidak dilakukan oleh orang yang berakal
- ✓ Area risiko besar bagi produsen perangkat lunak

Warranty

- ✓ Jaminan (*warranty*) menjamin pembeli atau penyewa bahwa suatu produk memenuhi standar kualitas tertentu
- ✓ Jika produk gagal memenuhi ketentuan garansi, pembeli atau penyewa dapat menuntut pelanggaran garansi (*breach of warranty*)

Intentional misrepresentation

- ✓ Penjual salah menggambarkan kualitas produk atau menyembunyikan cacat di dalamnya
- ✓ Bentuk representasi
  - Iklan
  - komentar Penjual
  - Faktur
  - Label pengiriman

Capability Maturity Model Integration

5

Capability Maturity Model Integration (CMMI)

- ✓ Pendekatan proses perbaikan
- ✓ Didefinisikan oleh Institut Rekayasa Perangkat Lunak
  - Di Universitas Carnegie Mellon di Pittsburgh
- ✓ Menentukan elemen penting dari proses yang efektif
- ✓ Cukup umum untuk mengevaluasi dan meningkatkan hampir semua proses
- ✓ Sering digunakan untuk menilai praktik pengembangan perangkat lunak

- ✓ Menentukan lima tingkat kematangan pengembangan perangkat lunak
- ✓ Mengidentifikasi masalah yang paling penting untuk kualitas perangkat lunak dan peningkatan proses
- ✓ Organisasi melakukan penilaian praktik pengembangan perangkat lunaknya
- ✓ Menentukan di mana mereka cocok dalam model kemampuan
- ✓ Mengidentifikasi area untuk perbaikan

- ✓ Pengembangan CMMI
  - Seperangkat pedoman untuk 22 area proses yang terkait dengan pengembangan sistem
  - Organisasi yang melakukan 22 hal ini dengan baik akan memiliki pengembangan perangkat lunak yang luar biasa

TABLE 7-2 Definition of CMMI maturity levels

Maturity level	Description
Initial	Process is ad hoc and chaotic; organization tends to overcommit and processes are often abandoned during times of crisis.
Managed	Projects employ processes and skilled people; status of work products is visible to management at defined points.
Defined	Processes are well defined and understood and are described in standards, procedures, tools, and methods; processes are consistent across the organization.
Quantitatively managed	Quantitative objectives for quality and process performance are established and are used as criteria in managing projects; specific measures of process performance are collected and statistically analyzed.
Optimizing	Organization continually improves its processes; changes are based on a quantitative understanding of its business objectives and performance needs.

Source: Used with permission from Carnegie Mellon University.

Development of Safety-Critical Systems

6

Safety-critical system

- ✓ Sistem yang kegagalannya dapat menyebabkan cedera/kematian
- ✓ Contoh
  - Rem anti penguncian mobil
  - Reaktor pembangkit listrik tenaga nuklir
  - Navigasi pesawat
  - Roller coaster
  - Elevator
  - Alat kesehatan

- ✓ Memerlukan proses pengembangan yang lebih ketat dan memakan waktu daripada jenis perangkat lunak lain
- ✓ Semua tugas membutuhkan:
    - Dokumentasi yang lebih menyeluruh
    - Pengecekan ulang yang waspada

- ✓ Project safety engineer
  - Tanggung jawab eksplisit untuk keamanan sistem
  - Menggunakan sistem logging dan pemantauan
- ✓ Hazard log
  - Menilai tim proyek bertanggung jawab untuk bahaya yang terdeteksi

- ✓ Risk
  - Konsekuensi meliputi:
    - Kerusakan properti
    - Kehilangan uang
    - Cedera pada orang
    - Kematian

- ✓ Redundancy
  - Penyediaan beberapa komponen yang mempunyai fungsi tunggal
  - Digunakan untuk mengatasi kegagalan dan kesalahan
  - Selama masa bencana yang meluas, kurangnya cadangan yang berlebihan dapat menyebabkan masalah besar

- ✓ N-version programming
  - Bentuk redundansi
  - Melibatkan pelaksanaan serangkaian instruksi program secara bersamaan oleh dua sistem yang berbeda
  - Menggunakan algoritma yang berbeda untuk menjalankan instruksi yang mencapai hasil yang sama

Quality Management Standards

7

ISO 9000 family

- ✓ The International Organization for Standardization (ISO), didirikan pada tahun 1947
- ✓ ISO mengeluarkan Standar ISO 9000 manajemen bisnis pada tahun 1988
- ✓ Standar ISO 9000 sebagai Panduan untuk produk, layanan, dan manajemen yang berkualitas
-

ISO 9000 family

- ✓ Seri standar ISO 9000 membahas :
  - ISO 9001: Desain, pengembangan, produksi, instalasi, service
  - ISO 9002: Produksi, instalasi, service
  - ISO 9003: Inspeksi dan pengujian
  - ISO 9000-3: Pengembangan, penyediaan, dan pemeliharaan perangkat lunak
  - ISO 9004: Manajemen mutu dan elemen sistem mutu

✓ Failure mode and effects analysis (FMEA)

- Teknik yang digunakan untuk mengevaluasi keandalan dan menentukan pengaruh kegagalan sistem dan peralatan

- Kegagalan diklasifikasikan berdasarkan:
    - Berdampak pada kesuksesan proyek
    - Keselamatan personel
    - Keamanan peralatan
    - Kepuasan dan keamanan pelanggan

- Tujuan
    - Identifikasi potensi kegagalan desain dan proses

TABLE 7-3 Manager's checklist for improving software quality

Question	Yes	No
Has senior management made a commitment to develop quality software?		
Have you used CMMI to evaluate your organization's software development process?		
Has your company adopted a standard software development methodology?		
Does the methodology place a heavy emphasis on quality management, and address how to define, measure, and refine the quality of the software development process and its products?		
Are software project managers and team members trained in the use of this methodology?		
Are software project managers and team members held accountable for following this methodology?		
Is a strong effort made to identify and remove errors as early as possible in the software development process?		

Activate Window